LAPORAN PRAKTIKUM ALGORITMA DAN STRUKTUR DATA

OLEH:

Nama	:Zafir Amal Maulana
Nim	:2025573010141
Kelas	: TI 1\C
Jurusan	: Teknologi Informasi dan Komputer
Dosen	: M. Reza Zulman S,ST.M.Sc

PROGRAM STUDI TEKNIK INFORMATIKA
JURUSAN TEKNOLOGI INFORMASI DAN KOMPUTER
POLITEKNIK NEGERI LHOKSEUMAWE
2026

LEMBAR PENGESAHAN

Nama Mata Kuliah	: Praktikum Algoritma Dan Struktu data
Judul Pratikum	: Praktikum Dasar-Dasar function pada JavaScript
Hari/Tanggal Pratikum	: 8 April 2026
Tanggal Penyerahan	: 14 April 2026
Tempat	: Laboratorium (TIK.106)
Nama Mahasiswa	: Zafir Amal Maulana
NIM	: 2025573010141
Program Studi/Kelas	: Teknik Informatika / TI.1C

Mengetahui
Dosen Pengajar,

M. Reza Zulman S,ST.M.Sc

DAFTAR ISI

LEMBAR PENGESAHAN ii

KATA PENGANTAR iv

BAB I PENDAHULUAN 1

1.2 Latar belakang 1

1.2 Tujuan Praktikum 1

BAB II LANDASAN TEORI 2

2.2 Kegunaan Function Javascript.....3

BAB III METODOLOGI PRAKTIKUM 4

3.1 Alat Dan Bahan 4

3.2 Langkah-langkah 4

BAB IV PENUTUP 24

4.1 KESIMPULAN 24

4.2 SARAN 25

KATA PENGANTAR

Puji syukur penulis panjatkan kepada Tuhan Yang Maha Esa atas segala rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan laporan praktikum yang berjudul “Praktikum Dasar Pemrograman Bahasa JavaScript” dengan baik dan tepat waktu. Laporan ini disusun sebagai salah satu bentuk pertanggungjawaban penulis dalam melaksanakan kegiatan praktikum serta untuk memenuhi tugas yang telah diberikan.

Laporan ini bertujuan untuk mempelajari dan memahami dasar-dasar pemrograman menggunakan JavaScript. Melalui praktikum ini, penulis dapat mengetahui konsep dasar JavaScript, seperti penggunaan variabel, tipe data, operator, struktur kontrol, serta fungsi dalam pemrograman. Selain itu, saya juga dapat memahami bagaimana JavaScript digunakan untuk membuat halaman web menjadi lebih interaktif dan dinamis.

Selain itu, praktikum ini juga memberikan pemahaman mengenai peran JavaScript sebagai bahasa pemrograman yang digunakan dalam pengembangan web, khususnya pada sisi client (client-side). Dengan mempelajari JavaScript, penulis dapat memahami bagaimana suatu aplikasi web dapat merespon input pengguna dan menjalankan logika program secara langsung di browser.

Saya berharap melalui praktikum ini dapat menambah wawasan dan keterampilan dalam bidang pemrograman, khususnya dalam memahami dasar-dasar bahasa JavaScript serta penerapannya dalam pengembangan web. Saya menyadari bahwa dalam penyusunan laporan ini masih terdapat kekurangan baik dari segi penulisan maupun isi materi yang disampaikan. Oleh karena itu, saya sangat mengharapkan kritik dan saran yang membangun dari berbagai pihak demi perbaikan dan penyempurnaan laporan ini di masa yang akan datang.

Akhir kata, saya berharap semoga laporan ini dapat memberikan manfaat, menambah wawasan, serta menjadi referensi bagi pembaca yang ingin mempelajari lebih lanjut mengenai dasar-dasar pemrograman menggunakan JavaScript.

BAB I
PENDAHULUAN

1.2 Latar belakang

Perkembangan teknologi informasi dan internet yang semakin pesat memberikan pengaruh besar dalam berbagai bidang kehidupan, terutama dalam pengembangan aplikasi berbasis web. Website tidak hanya digunakan sebagai media penyampaian informasi, tetapi juga sebagai sarana interaksi yang dinamis bagi pengguna. Oleh karena itu, diperlukan bahasa pemrograman yang mampu mendukung interaktivitas tersebut, salah satunya adalah JavaScript.

JavaScript merupakan bahasa pemrograman yang berperan penting dalam pengembangan web karena dapat menambahkan fungsi logika dan interaksi pada halaman web, seperti pengolahan data, validasi input, serta manipulasi tampilan secara langsung di browser. Salah satu konsep penting dalam JavaScript adalah ***function*** (fungsi), yaitu blok kode yang dirancang untuk menjalankan tugas tertentu dan dapat digunakan kembali. Penggunaan function membantu dalam menyusun kode yang lebih terstruktur, efisien, dan mudah dipelihara.

Melalui praktikum dasar function JavaScript, peserta diharapkan dapat memahami konsep dan penerapan fungsi dalam pemrograman. Materi yang dipelajari meliputi pembuatan dan pemanggilan fungsi, parameter dan argumen, nilai kembalian (*return value*), serta berbagai jenis fungsi seperti ***function declaration, function expression*, dan *arrow function*.** Selain itu, praktikum ini juga bertujuan untuk melatih kemampuan logika dan pemecahan masalah dalam membuat program sederhana yang dapat dijalankan pada browser.

Dengan adanya praktikum ini, diharapkan peserta mampu mengimplementasikan fungsi JavaScript secara efektif dalam pengembangan aplikasi web yang interaktif dan dinamis.

1.2 Tujuan Praktikum

1. Memahami konsep dasar JavaScriptsebagai bahasa pemrograman yang digunakan untuk menambahkan interaktivitas pada halaman web.
2. Memahami konsep fungsi (*function*) dalam JavaScript sebagai blok kode yang digunakan untuk menjalankan tugas tertentu.
3. Memahami cara menggunakan fungsi seperti *function declaration*, *function expression*, dan *arrow function*.

4. Mengetahui cara menggunakan nilai kembalian **(***return value***)** untuk menghasilkan output dari suatu fungsi.
5. Melatih keterampilan pemrograman yang terstruktur, efisien, dan mudah dipelihara dalam pengembangan aplikasi berbasis web.

BAB II
LANDASAN TEORI

2.1 **PENGERTIAN JAVASCRIPT DAN FUNCTION**

Praktikum Dasar Pemrograman JavaScript merupakan kegiatan pembelajaran yang bertujuan untuk memahami konsep dan penerapan bahasa pemrograman JavaScript dalam pengembangan web. Seiring dengan pesatnya perkembangan teknologi informasi dan internet, JavaScript menjadi salah satu bahasa pemrograman utama yang digunakan untuk menciptakan halaman web yang interaktif, dinamis, dan responsif. Oleh karena itu, pemahaman terhadap dasar-dasar JavaScript sangat penting bagi mahasiswa atau siswa yang mempelajari bidang teknologi informasi, khususnya dalam menghadapi tuntutan dunia digital yang semakin berkembang.

JavaScript adalah bahasa pemrograman yang berjalan di sisi klien (*client-side*) dan dapat langsung dijalankan melalui web browser seperti Google Chrome, Mozilla Firefox, dan Microsoft Edge tanpa memerlukan proses kompilasi tambahan. Bahasa ini digunakan untuk menambahkan interaktivitas pada halaman web, seperti memproses data, memvalidasi input pengguna, menampilkan pesan, membuat animasi sederhana, serta memanipulasi elemen HTML dan CSS secara langsung. Dengan kemampuannya tersebut, JavaScript menjadi komponen penting dalam pengembangan website modern.

Dalam praktikum ini,saya mempelajari berbagai konsep dasar pemrograman JavaScript, antara lain variabel, tipe data, operator, percabangan, perulangan, serta manipulasi Document Object Model (DOM). Selain itu, peserta juga akan memahami cara mengintegrasikan JavaScript dengan HTML dan CSS untuk menghasilkan tampilan web yang lebih menarik dan fungsional. Pemahaman ini akan membantu peserta dalam membangun aplikasi web sederhana secara mandiri. Salah satu materi utama dalam praktikum ini adalah *function* (fungsi). Function merupakan sekumpulan perintah yang dirancang untuk menjalankan tugas tertentu dan dapat digunakan kembali (*reusable*). Penggunaan fungsi memungkinkan kode program menjadi lebih terstruktur, efisien, mudah dipahami, serta mempermudah proses pengembangan dan pemeliharaan aplikasi. Dalam JavaScript, fungsi dapat

dibuat dalam berbagai bentuk, seperti ***function declaration, function expression,*** dan ***arrow function,*** yang masing-masing memiliki kelebihan tersendiri.

Melalui praktikum dasar pemrograman JavaScript dan function, peserta diharapkan mampu memahami konsep dasar pemrograman, mengembangkan logika berpikir yang sistematis, serta meningkatkan kemampuan dalam memecahkan masalah. Dengan penguasaan materi ini, peserta akan memiliki landasan yang kuat untuk mempelajari teknologi web yang lebih lanjut dan mengembangkan aplikasi berbasis web yang interaktif, dinamis, dan profesional

2.2 Kegunaan Function Javascript

Fungsi (*Function*) dalam JavaScript berperan untuk menjalankan perintah tertentu serta membantu menyusun program agar lebih terstruktur, efisien, dan mudah dipahami. Fungsi juga memiliki kegunaan antara lain:

- Mengurangi pengulangan kode.
- Membuat program lebih terstruktur dan efisien.
- Mempermudah pemeliharaan dan pengembangan.
- Membantu dalam pemecahan masalah.
- Mengolah data melalui parameter dan nilai kembalian.

BAB III
METODOLOGI PRAKTIKUM

3.1 Alat Dan Bahan

- PC/Laptop
- Koneksi internet (opsional)
- VScode
- Bahasa Pemrograman JavaScript
- Panduan Praktikum (Digunakan untuk memahami materi dan menyelesaikan tugas)

3.2 Langkah-langkah

1. buka Gitbash Kemudian navigasikan ke folder repository GitHub kamu yang sudah di-clone.buat folder baru Bernama "**paktikum-3**" untuk menyimpan semua file. lalu ketik "**mkdir praktikum-3**" enter, lalu ketik "**cd praktikum-3**"enter.lalu masuk ke dalam folder tersebut.

   (pastikan kamu berada di folder modul_3) lalu ketik prompt berikut;

   "touch 01-function-dasar.js"

   "touch 02-arrow-callback.js"

   "touch 03-array-dasar.js"

   "node 03-array-dasar.js"

   "touch 04-array-method.js"

   "node 04-array-method.js"

   Setelah membuat file" tersebut ketikkan kode" berikut sesuai dengan intruksi metode praktikum/modul praktikum.

A. “ 01-function-dasar.js”

Function adalah blok kode yang diberi nama dan dapat dipanggil (dieksekusi) kapan saja dan berulang kali. Bayangkan function seperti sebuah mesin atau resep kamu mendefinisikannya sekali, lalu bisa menggunakannya berkali-kali tanpa perlu menulis ulang kodenya. Mengapa function penting? Prinsip utama pemrograman adalah DRY (Don't Repeat Yourself). Daripada menulis kode yang sama di banyak tempat, cukup bungkus kode itu dalam sebuah function.Anatomi sebuah function:

- Parameter — Variabel input yang diterima function dari luar. Didefinisikan di dalam kurung ().
- Return value — Nilai yang dikembalikan function setelah selesai dieksekusi, menggunakan kata kunci return.
- Scope — Variabel yang dideklarasikan di dalam function hanya bisa diakses di dalam function itu saja (local scope). Variabel di luar function disebut global scope. Perlu diingat: setelah JavaScript menemukan kata kunci return, eksekusi function langsung berhenti. Kode di bawah return tidak akan pernah dijalankan.

  Ketik code dibawah ini sesuai intruksi yang di modul.lalu jalankan dan amati hasil dari code tersebut..

  Function sapa() dipanggil dua kali dan mencetak output yang sama ini mendemonstrasikan penggunaan ulang (reusability).

  Function tambah() mengembalikan nilai yang bisa disimpan ke variabel atau langsung digunakan.

  Perhatikan **hitung(5**) menggunakan nilai default pengali = 2, sedangkan hitung**(5, 3)** menimpa default dengan nilai 3.

Simpan file code tersebut lalu jalanakan di terminal**” node 01-function-dasar.js”**

```javascript
function sapa() {
  console.log("Halo! Selamat datang di praktikum JavaScript.");
}
sapa();
sapa();
function sapaNama(nama) {
  console.log(`Halo, ${nama}! Selamat belajar.`);
}
sapaNama("Budi");
sapaNama("Siti");

function tambah(angka1, angka2) {
  const hasil = angka1 + angka2;
  return hasil;
}
const hasilPenjumlahan = tambah(10, 25);
console.log("10 + 25 =", hasilPenjumlahan);
console.log("7 + 13 =", tambah(7, 13));

function hitung(nilai, pengali = 2) {
  return nilai * pengali;
}
console.log(hitung(5));
console.log(hitung(5, 3));
const pesanGlobal = "Saya ada di mana saja";
function cekScope() {
  const pesanLokal = "Saya hanya ada di dalam function ini"; // local scope
  console.log(pesanGlobal);
  console.log(pesanLokal);
}
cekScope();
console.log(pesanGlobal);


//LATIHAN NO 1//
function tambah(a, b) {
  return a + b;
}
function kurang(a, b) {
  return a - b;
}
function perkalian(a, b) {
  return a * b;
}
function pembagian(a, b) {
  if (b === 0) {
    console.log("error:tidak bisa dibagi dengan nol");
    return null;
  }
  return a / b;
}
function kalkulator(a, operator, b) {
  switch (operator) {
    case "+":
      return tambah(a, b);
    case "-":
      return kurang(a, b);
    case "*":
      return perkalian(a, b);
    case "/":
      return pembagian(a, b);
    default:
      console.log("operator tidak ditemukan");
      return null;
  }
}
console.log("10+15 =", kalkulator(10, "+", 15));
console.log("100-30 =", kalkulator(100, "-", 30));
console.log("25*55 =", kalkulator(25, "*", 55));
console.log("10/2=", kalkulator(10, "/", 2));
console.log("20/0=", kalkulator(20, "/", 0));
```

```
PS C:\2025573010141_praktikum_algoritma_struktur_data\praktikum-3
> node 01-function-dasar.js
Halo! Selamat datang di praktikum JavaScript.
Halo! Selamat datang di praktikum JavaScript.
Halo, Budi! Selamat belajar.
Halo, Siti! Selamat belajar.
10 + 25 = 35
7 + 13 = 20
10
15
Saya ada di mana saja
Saya hanya ada di dalam function ini
Saya ada di mana saja
10+15 = 25
100-30 = 70
25*55 = 1375
10/2= 5
error:tidak bisa dibagi dengan nol
20/0= null
PS C:\2025573010141_praktikum_algoritma_struktur_data\praktikum-3
>
```

B. “02-arrow-callback.js”

Arrow Function dan Konsep Callback

Arrow function adalah sintaks penulisan function yang lebih ringkas, diperkenalkan di ES6 (2015). Alih-alih menulis **function nama(params){... },** kita bisa menulis **const nama = (params) => { ... }.**

Aturan penulisan arrow function:

▪Jika hanya ada satu parameter, kurung bisa dihilangkan: **x => x * 2**

▪Jika body hanya satu baris dan langsung return, kurung kurawal dan return bisa dihilangkan: (a, b) => a + b

▪Jika body lebih dari satu baris, tetap gunakan kurung kurawal dan return.

Callback adalah sebuah function yang dikirimkan sebagai argumen ke function lain.Function penerima akan memanggil callback tersebut di waktu yang tepat. Ini adalah konsep yang sangat fundamental di JavaScript hampir semua event handling, timer, dan method array menggunakan callback.

Buka file 02-arrow-callback.js di VS Code, lalu ketikkan kode berikut.dan Simpan file, lalu jalankan dari terminal. Dengan menggunakan prompt**" node 02-arrow-callback.js"**

```javascript
function kuadratBiasa(x) {
  return x * x;
}

const kuadrat = (x) => {
  return x * x;
};
const kuadratRingkas = (x) => x * x;
console.log("=== Perbandingan Penulisan ===");
console.log("Biasa :", kuadratBiasa(5));
console.log("Arrow :", kuadrat(5)); //
console.log("Ringkas :", kuadratRingkas(5));

const luas = (panjang, lebar) => panjang * lebar;
const salam = (nama, waktu) => `Selamat ${waktu}, ${nama}!`;
console.log("Luas 4x6 :", luas(4, 6));
console.log(salam("Budi", "Pagi"));

function lakukanOperasi(angka, operasiCallback) {
  console.log(`Angka awal: ${angka}`);
  const hasil = operasiCallback(angka);
  console.log(`Hasil setelah operasi: ${hasil}`);
}
console.log("\n=== Callback ===");

lakukanOperasi(7, kuadratRingkas);
lakukanOperasi(10, (x) => x + 100);
lakukanOperasi(20, function (x) {
  return x / 2;
});
console.log("\n=== setTimeout (Callback) ===");
console.log("Pesan 1: Sebelum timer");
setTimeout(() => {
  console.log("Pesan 3: Ini dari dalam setTimeout (setelah 1 detik)");
}, 1000);
console.log("Pesan 2: Setelah mendaftarkan timer");

/// LATIHAN 2 ///

const keHurufBesar = (teks) => teks.toUpperCase();
const tambahSeru = (teks) => teks + "!!";
const hitungKata = (teks) => teks.split(" ").length;
function prosesKalimat(kalimat, transformasiCallback) {
  const hasil = transformasiCallback(kalimat);
  console.log(hasil);
  return hasil;
}

const kalimat = "belajar javascript itu menyenangkan";

console.log("Kalimat Asli:", kalimat);
console.log("\nHasil Transformasi:");

prosesKalimat(kalimat, keHurufBesar);
prosesKalimat(kalimat, tambahSeru);
prosesKalimat(kalimat, hitungKata);
```

```
PS C:\2025573010141_praktikum_algoritma_struktur_data\praktikum-3> node 02-arrow-callback.js
=== Perbandingan Penulisan ===
Biasa : 25
Arrow : 25
Ringkas : 25
Luas 4x6 : 24
Selamat Pagi, Budi!

=== Callback ===
Angka awal: 7
Arrow : 25
Ringkas : 25
Luas 4x6 : 24
Selamat Pagi, Budi!
```

```
=== Callback ===
Angka awal: 7

=== Callback ===
Angka awal: 7
Angka awal: 7
Hasil setelah operasi: 49
Angka awal: 10
Hasil setelah operasi: 110
Angka awal: 20
Hasil setelah operasi: 10

=== setTimeout (Callback) ===
Pesan 1: Sebelum timer
Pesan 2: Setelah mendaftarkan timer
Kalimat Asli: belajar javascript itu menyenangkan

Hasil Transformasi:
BELAJAR JAVASCRIPT ITU MENYENANGKAN
belajar javascript itu menyenangkan!!
4
```

C. “03-array-dasar.js”

Array adalah struktur data yang menyimpan kumpulan nilai dalam satu variabel, berurutan dan terindeks. Kita bisa membayangkan array seperti sebuah rak buku bernomor setiap buku (elemen) memiliki posisi (indeks) yang dimulai dari angka 0.

Contoh**: const buah = ['Apel', 'Mangga', 'Jeruk']**

▪**buah[0] :** 'Apel' (indeks pertama = 0)

▪**buah[1] :** 'Mangga' (indeks kedua = 1)

▪**buah[2] :** 'Jeruk' (indeks ketiga = 2)

▪ **buah.length :** 3 (jumlah elemen)

Array di JavaScript bersifat dinamis — ukurannya bisa berubah sewaktu-waktu. Array juga bisa menyimpan tipe data yang berbeda-beda dalam satu array (meskipun praktik terbaik adalah menyimpan satu jenis data per array).

Method bawaan array yang sering digunakan untuk manipulasi:

▪**push(x) :**tambah elemen x di akhir array

▪**pop() :**hapus dan kembalikan elemen terakhir

▪**unshift(x):** tambah elemen x di awal array

▪**shift() :** hapus dan kembalikan elemen pertama

▪**splice(i, n) :**hapus n elemen mulai indeks i

▪**slice(a, b) :**salin elemen dari indeks a sampai sebelum b (non destructive)

▪**indexOf(x) :**cari indeks elemen x (-1 jika tidak ditemukan)

▪**includes(x) :** cek apakah x ada di array (mengembalikan boolean)

Selanjutnya ketikkan Code berikut ini, jalankan**"node 03-array-dasar.js"** dan amati hasilnya;

```javascript
const mahasiswa = ["budi", "siti", "ahmad", "rina"];
const nilai = [85, 92, 78, 88];

console.log("=== Array Awal ===");
console.log("Mahasiswa:", mahasiswa);
console.log("Nilai :", nilai);
console.log("Jumlah mahasiswa:", mahasiswa.length);

console.log("\n=== Akses Elemen ===");
console.log("Elemen pertama :", mahasiswa[0]);
console.log("Elemen ketiga :", mahasiswa[2]);
console.log("Elemen terakhir:", mahasiswa[mahasiswa.length - 1]);

mahasiswa[1] = "Siti Rahayu";
console.log("\nSetelah diubah:", mahasiswa);

console.log("\n=== Manipulasi Array ===");
mahasiswa.push("Doni");
console.log("Setelah push :", mahasiswa);
mahasiswa.unshift("Zahra");
console.log("Setelah unshift :", mahasiswa);
const dihapusAkhir = mahasiswa.pop();
console.log("Dihapus (pop) :", dihapusAkhir);
console.log("Setelah pop :", mahasiswa);
const dihapusAwal = mahasiswa.shift();
console.log("Dihapus (shift) :", dihapusAwal);
console.log("Setelah shift :", mahasiswa);

console.log("\n=== Pencarian ===");
console.log("Indeks Ahmad :", mahasiswa.indexOf("Ahmad"));
console.log("Ada Rina? :", mahasiswa.includes("Rina"));
console.log("Ada Budi? :", mahasiswa.includes("Budi"));

const sebagian = nilai.slice(1, 4);
console.log("\nnilai asli :", nilai);
console.log("slice[1,4]:", sebagian);

//LATIHAN 3 //
let daftarBelanja = ["Beras", "Gula", "Minyak", "silverqueen", "kopi"];

console.log("Daftar Belanja:");
for (let i = 0; i < daftarBelanja.length; i++) {
  console.log(`${i + 1}. ${daftarBelanja[i]}`);
}

daftarBelanja.push("Susu", "garam");
console.log("\nSetelah ditambah item:");
console.log(daftarBelanja);

let itemDihapus = daftarBelanja.shift();
console.log(`\nItem yang dihapus: ${itemDihapus}`);

let adaSusu = daftarBelanja.includes("Susu");
console.log(`Apakah ada 'Susu'? ${adaSusu ? "Ya, ada" : "Tidak ada"}`);

console.log(`Jumlah item akhir: ${daftarBelanja.length}`);
```

```
PS C:\2025573010141_praktikum_algoritma_struktur_data\praktikum-3> node 03-array-dasar.js
=== Array Awal ===
Mahasiswa: [ 'budi', 'siti', 'ahmad', 'rina' ]
Nilai : [ 85, 92, 78, 88 ]
Jumlah mahasiswa: 4

=== Akses Elemen ===
Elemen pertama : budi
Elemen ketiga : ahmad
Elemen terakhir: rina

Setelah diubah: [ 'budi', 'Siti Rahayu', 'ahmad', 'rina' ]

=== Manipulasi Array ===
Setelah push : [ 'budi', 'Siti Rahayu', 'ahmad', 'rina', 'Doni' ]
Setelah unshift : [ 'Zahra', 'budi', 'Siti Rahayu', 'ahmad', 'rina', 'Doni' ]
Dihapus (pop) : Doni
Setelah pop : [ 'Zahra', 'budi', 'Siti Rahayu', 'ahmad', 'rina' ]
Dihapus (shift) : Zahra
Setelah shift : [ 'budi', 'Siti Rahayu', 'ahmad', 'rina' ]

=== Pencarian ===
Indeks Ahmad : -1
Ada Rina? : false
Ada Budi? : false

nilai asli : [ 85, 92, 78, 88 ]
slice[1,4]: [ 92, 78, 88 ]
 Daftar Belanja:
 2. Gula
 3. Minyak
 4. silverqueen
 5. kopi

 Setelah ditambah item:
 [ 'Beras', 'Gula', 'Minyak', 'silverqueen', 'kopi', 'Susu', 'garam' ]

 Item yang dihapus: Beras
 Apakah ada 'Susu'? Ya, ada
 Jumlah item akhir: 6
 PS C:\2025573010141_praktikum_algoritma_struktur_data\praktikum-3>
```

D. “04-array-method.js”

Method Iterasi Array yang Modern

JavaScript menyediakan method iterasi array yang powerful. Method-method ini menerima sebuah callback function sebagai argumen, dan secara otomatis memanggil callback untuk setiap elemen array.

forEach(callback): Menjalankan callback untuk setiap elemen. Tidak mengembalikan nilai apapun. Digunakan hanya untuk efek samping (side effects) seperti mencetak atau menyimpan ke database.

map(callback):Membuat array baru dengan menerapkan callback ke setiap elemen. Array asli tidak berubah. Digunakan untuk mentransformasi setiap elemen.

filter(callback) : Membuat array baru yang hanya berisi elemen yang memenuhi kondisi (callback mengembalikan **true**). Array asli tidak berubah. Digunakan untuk menyaring elemen.

reduce(callback, nilaiAwal): Mereduksi seluruh array menjadi satu nilai tunggal.Callback menerima dua argumen: **accumulator** (nilai yang terakumulasi) dan **currentValue** (elemen saat ini).

Note: **map** dan **filter** selalu menghasilkan array baru, sedangkan **reduce** menghasilkan satu nilai (angka, string, object, dll).

Ketikkan kode berikut **“node 04-array-method.js”** dan liat hasilnya.

```javascript
const nilaimahasiswa = [75, 90, 55, 82, 68, 95, 48, 77];

console.log("=== forEach: Tampilkan Semua Nilai ===");
nilaimahasiswa.forEach((nilai, indeks) => {
  console.log(` Mahasiswa-${indeks + 1}: ${nilai}`);
});

const gradeHuruf = nilaimahasiswa.map((nilai) => {
  if (nilai >= 90) return "A";
  if (nilai >= 80) return "B";
  if (nilai >= 70) return "C";
  if (nilai >= 60) return "D";
  return "E";
});
console.log("\n=== map: Nilai ke Grade ===");
console.log("Nilai :", nilaimahasiswa);
console.log("Grade :", gradeHuruf);

const nilaiLulus = nilaimahasiswa.filter((nilai) => nilai >= 60);
const nilaiGagal = nilaimahasiswa.filter((nilai) => nilai < 60);
console.log("\n=== filter: Lulus dan Tidak Lulus ===");
console.log("Semua nilai :", nilaimahasiswa);
console.log("Nilai lulus :", nilaiLulus);
console.log("Nilai gagal :", nilaiGagal);

const totalNilai = nilaimahasiswa.reduce((akumulator, nilai) => {
  return akumulator + nilai;
}, 0);
const rataRata = totalNilai / nilaimahasiswa.length;
console.log("\n=== reduce: Statistik Nilai ===");
console.log("Total nilai :", totalNilai);
console.log("Rata-rata :", rataRata.toFixed(2));

const rataratanilailulus = nilaimahasiswa
  .filter((nilai) => (nilai) => 60)
  .reduce((acc, val, idx, arr) => {
    return acc + val / arr.length;
  }, 0);

console.log("\n=== method chaining ===");
console.log("rata-rata nilai lulus:", rataratanilailulus.toFixed(2));

// LATIHAN 4 //

const produk = [
  { nama: "Laptop", harga: 8500000, stok: 5 },
  { nama: "Mouse", harga: 150000, stok: 0 },
  { nama: "Keyboard", harga: 450000, stok: 12 },
  { nama: "Monitor", harga: 3200000, stok: 0 },
  { nama: "Headset", harga: 350000, stok: 8 },
];

const formatRupiah = (angka) => {
  return "Rp" + angka.toLocaleString("id-ID");
};

console.log("=== Analisis Data Produk ===\n");

const produktersedia = produk.filter((item) => item.stok > 0);
console.log("Produk Tersedia:");
console.log(produktersedia);
console.log();

console.log("Nama Produk yang Tersedia:");
console.log(produktersedia);
console.log();

const totalInventaris = produktersedia.reduce(
  (total, item) => total + item.harga * item.stok,
  0,
);
console.log("Total Nilai Inventaris:", formatRupiah(totalInventaris));
console.log();

console.log("Daftar Produk:");
produk.forEach((item) => {
  if (item.stok > 0) {
    console.log(
      `[TERSEDIA] ${item.nama} - ${formatRupiah(item.harga)} (${item.stok} unit)`,
    );
  } else {
    console.log(`[HABIS] ${item.nama} - ${formatRupiah(item.harga)}`);
  }
});
```

```
PS C:\2025573010141_praktikum_algoritma_struktur_data\praktikum-3> node 04-array-method.js
=== forEach: Tampilkan Semua Nilai ===
 Mahasiswa-1: 75
 Mahasiswa-2: 90
 Mahasiswa-3: 55
 Mahasiswa-4: 82
 Mahasiswa-5: 68
 Mahasiswa-6: 95
 Mahasiswa-7: 48
 Mahasiswa-8: 77

=== map: Nilai ke Grade ===
Nilai : [
  75, 90, 55, 82,
  68, 95, 48, 77
]
Grade : [
  'C', 'A', 'E',
  'B', 'D', 'A',
  'E', 'C'
]

=== filter: Lulus dan Tidak Lulus ===
Semua nilai : [
  75, 90, 55, 82,
  68, 95, 48, 77
]
Nilai lulus : [ 75, 90, 82, 68, 95, 77 ]
Nilai gagal : [ 55, 48 ]

=== reduce: Statistik Nilai ===
Total nilai : 590
Rata-rata : 73.75

=== method chaining ===
rata-rata nilai lulus: 73.75
```

```
=== Analisis Data Produk ===

Produk Tersedia:
[
  { nama: 'Laptop', harga: 8500000, stok: 5 },
  { nama: 'Keyboard', harga: 450000, stok: 12 },
  { nama: 'Headset', harga: 350000, stok: 8 }
]

Nama Produk yang Tersedia:
[
  { nama: 'Laptop', harga: 8500000, stok: 5 },
  { nama: 'Keyboard', harga: 450000, stok: 12 },
  { nama: 'Headset', harga: 350000, stok: 8 }
]

Total Nilai Inventaris: Rp50.700.000

Daftar Produk:
[TERSEDIA] Laptop - Rp8.500.000 (5 unit)
[HABIS] Mouse - Rp150.000
[TERSEDIA] Keyboard - Rp450.000 (12 unit)
[HABIS] Monitor - Rp3.200.000
[TERSEDIA] Headset - Rp350.000 (8 unit)
PS C:\2025573010141_praktikum_algoritma_struktur_data\praktikum-3>
```

E. “05-rekursi.js”

Rekursi adalah teknik di mana sebuah function memanggil dirinya sendiri untuk memecahkanmasalah yang lebih kecil secara bertahap, sampai mencapai kondisi berhenti.

Setiap function rekursif harus memiliki dua bagian:

Base case (kondisi berhenti) : kondisi di mana function tidak lagi memanggil dirinya,sendiri. Tanpa ini, rekursi akan berjalan selamanya dan menyebabkan error Maximum call “**stack exceeded”.**

Recursive case : bagian di mana function memanggil dirinya sendiri dengan input yang lebih kecil/sederhana.

Contoh: Menghitung faktorial dari n (n!):

▪5! = 5 × 4! = 5 × 4 × 3! = 5 × 4 × 3 × 2! = 5 × 4 × 3 × 2 × 1! = 120

▪Base case: faktorial(1) = 1

▪Recursive case: faktorial(n) = n × faktorial(n-1)

Rekursi sangat penting dalam Struktur Data dan Algoritma karena banyak algoritma pada pohon

(tree), graf (graph), dan pencarian (searching) yang paling mudah diimplementasikan secara rekursif.

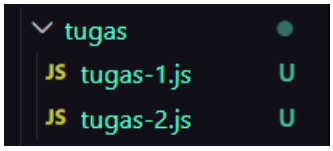
Buka file **05-rekursi.js** di VS Code, lalu ketikkan kode ini dan jalankan di terminal**"node 05-rekursi.js"**lalu **amati hasilnya.**.

```javascript
function faktorial(n) {
  if (n <= 1) return 1;

  return n * faktorial(n - 1);
}
console.log("=== Faktorial ===");
console.log("0! =", faktorial(0));
console.log("1! =", faktorial(1));
console.log("5! =", faktorial(5));
console.log("7! =", faktorial(7));

function fibonacci(n) {
  // Base case
  if (n === 0) return 0;
  if (n === 1) return 1;

  return fibonacci(n - 1) + fibonacci(n - 2);
}
console.log("\n=== Fibonacci ===");
for (let i = 0; i <= 8; i++) {
  process.stdout.write(fibonacci(i) + " ");
}
console.log("");

function jumlahArray(arr, indeks = 0) {
  if (indeks === arr.length) return 0;

  return arr[indeks] + jumlahArray(arr, indeks + 1);
}
const angka = [3, 7, 2, 9, 5];
console.log("\n=== Jumlah Array dengan Rekursi ===");
console.log("Array :", angka);
console.log("Jumlah :", jumlahArray(angka)); // 26

function countdown(n) {
  if (n < 0) {
    console.log("Selesai!");
    return;
  }
  console.log(n);
  countdown(n - 1);
}
console.log("\n=== Countdown Rekursif ===");
countdown(5);
```

```
PS C:\2025573010141_praktikum_algoritma_struktur_data> cd C:\2025573010141_praktikum_algoritma_struktur_data\praktikum-3
PS C:\2025573010141_praktikum_algoritma_struktur_data\praktikum-3> node 05-rekursi.js
=== Faktorial ===
0! = 1
1! = 1
5! = 120
7! = 5040

=== Fibonacci ===
0 1 1 2 3 5 8 13 21

=== Jumlah Array dengan Rekursi ===
Array : [ 3, 7, 2, 9, 5 ]
Jumlah : 26

=== Countdown Rekursif ===
5
4
3
2
1
0
Selesai!
PS C:\2025573010141_praktikum_algoritma_struktur_data\praktikum-3>
```

Selanjutnya buka terminal buat folder baru berisi 2 file **"tugas-1.js" dan "tugas-2.js"** di dalam praktikum-3 setelah itu ketik code sesuai intruksi pada modul.

```
mkdir tugas
touch tugas/tugas-1.js tugas/tugas-2.js
```



F. “tugas-1.js”

Ketika sudah mengetik code yang sudah cari sesuai intruksi, jalankan di terminal.”node tugas-1.js”

```javascript
const dataMahasiswa = [
  { nama: "zafir", nilai: 80 },
  { nama: "nopal", nilai: 75 },
  { nama: "riyan", nilai: 80 },
  { nama: "daniel", nilai: 90 },
  { nama: "donadoni", nilai: 70 },
  { nama: "ridwan", nilai: 85 },
];
function hitungStatistik(arrMahasiswa) {
  const total = arrMahasiswa.length;
  const sum = arrMahasiswa.reduce((acc, mhs) => acc + mhs.nilai, 0);
  const rataRata = sum / total;

  const nilaiList = arrMahasiswa.map((mhs) => mhs.nilai);
  const tertinggi = Math.max(...nilaiList);
  const terendah = Math.min(...nilaiList);

  return {
    total,
    rataRata,
    tertinggi,
    terendah,
  };
}
function filterLulus(arrMahasiswa, batasLulus) {
  return arrMahasiswa.filter((mhs) => mhs.nilai >= batasLulus);
}
function tambahGrade(arrMahasiswa) {
  return arrMahasiswa.map((mhs) => {
    let grade = "";

    if (mhs.nilai >= 85) grade = "A";
    else if (mhs.nilai >= 75) grade = "B";
    else if (mhs.nilai >= 65) grade = "C";
    else if (mhs.nilai >= 50) grade = "D";
    else grade = "E";

    return {
      ...mhs,
      grade,
    };
  });
}

const statistik = hitungStatistik(dataMahasiswa);
const mahasiswaLulus = filterLulus(dataMahasiswa, 75);
const mahasiswaDenganGrade = tambahGrade(dataMahasiswa);

console.log("=== Statistik ===");
Object.entries(statistik).forEach(([key, value]) => {
  console.log(`${key}: ${value}`);
});

console.log("\n=== Mahasiswa Lulus ===");
mahasiswaLulus.forEach((mhs) => {
  console.log(`Nama: ${mhs.nama}, Nilai: ${mhs.nilai}`);
});

console.log("\n=== Mahasiswa dengan Grade ===");
mahasiswaDenganGrade.forEach((mhs) => {
  console.log(`Nama: ${mhs.nama}, Nilai: ${mhs.nilai}, Grade: ${mhs.grade}`);
});
```

```
Node.js v24.14.0
PS C:\2025573010141_praktikum_algoritma_struktur_data\praktikum-3\tugas> node tugas-1.js
=== Statistik ===
total: 6
rataRata: 80
tertinggi: 90
terendah: 70

=== Mahasiswa Lulus ===
Nama: zafir, Nilai: 80
Nama: nopal, Nilai: 75
Nama: riyan, Nilai: 80
Nama: daniel, Nilai: 90
Nama: ridwan, Nilai: 85

=== Mahasiswa dengan Grade ===
Nama: zafir, Nilai: 80, Grade: B
Nama: nopal, Nilai: 75, Grade: B
Nama: riyan, Nilai: 80, Grade: B
Nama: daniel, Nilai: 90, Grade: A
Nama: donadoni, Nilai: 70, Grade: C
Nama: ridwan, Nilai: 85, Grade: A
```

G. “tugas-2.js”

Jalankan di terminal dengan prompt”node tugas-2.js” dan liat hasilnya .

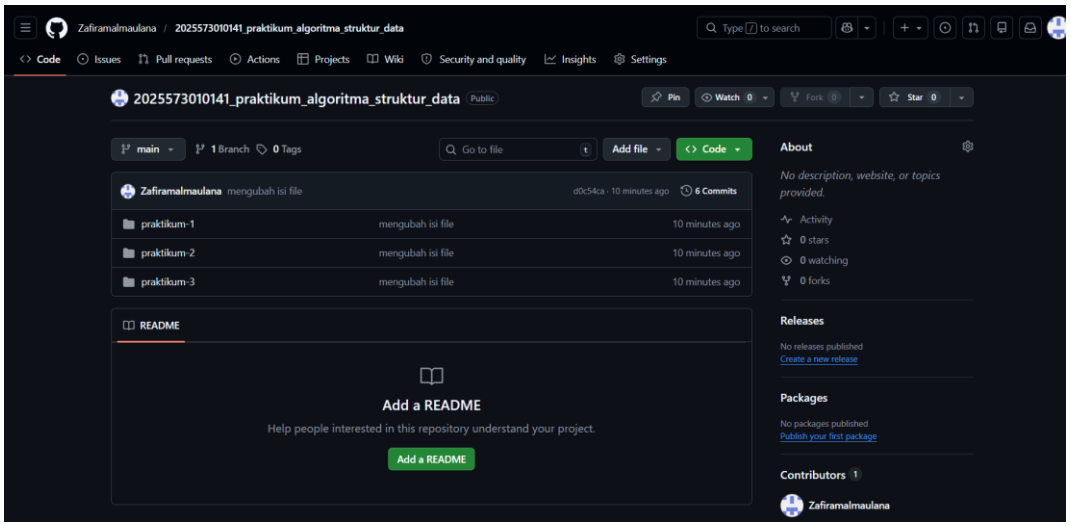
```javascript
function pangkat(basis, eksponen) {
  if (eksponen === 0) return 1;

  return basis * pangkat(basis, eksponen - 1);
}
function balikstring(str) {
  if (str === "" || str.lenght === 1) return str;

  return str[str.length - 1] + balikstring(str.slice(0, str.lenght - 1));
}
function cekpalindrom(str) {
  const hasilbalik = balikstring(str);
  return str === hasilbalik;
}

console.log("===pangkat====");
console.log("2^3= ${pangkat(2,3)}");
console.log("5^2= ${pangkat(5,2)}");
console.log("7^0 =${pangkat(7,0)}");

console.log("\n=== balik string ===");
console.log(`halo -> ${balikstring("halo")}`);
console.log(`data -> ${balikstring("data")}`);

console.log("\n=== cek palindrom ===");
const katalist = ["katak", "civic", "level", "halo"];

katalist.forEach((kata) => {
  console.log(
    '${kata} -> ${cekpalindrom(kata) ? "palindrom" : "bukan palindrom"}',
  );
});
```

```
PS C:\2025573010141_praktikum_algoritma_struktur_data\praktikum-3\tugas> node tugas-2.js
===pangkat====
2^3= ${pangkat(2,3)}
5^2= ${pangkat(5,2)}
7^0 =${pangkat(7,0)}

=== balik string ===
halo -> o
data -> a

=== cek palindrom ===
${kata} -> ${cekpalindrom(kata) ? "palindrom" : "bukan palindrom"}
${kata} -> ${cekpalindrom(kata) ? "palindrom" : "bukan palindrom"}
${kata} -> ${cekpalindrom(kata) ? "palindrom" : "bukan palindrom"}
${kata} -> ${cekpalindrom(kata) ? "palindrom" : "bukan palindrom"}
PS C:\2025573010141_praktikum_algoritma_struktur_data\praktikum-3\tugas>
```

Ketika sudah berhasil menyelesaikan Langkah – Langkah tersebut, selanjutnya memindahkan ke github maka tampilannya akan muncul seperti ini

```
PS C:\2025573010141_praktikum_algoritma_struktur_data\praktikum-3> git add .
PS C:\2025573010141_praktikum_algoritma_struktur_data\praktikum-3> git commit -m  "mengubah isi file"
[main d0c54ca] mengubah isi file
 14 files changed, 419 insertions(+), 4 deletions(-)
 rename README.md => praktikum-1/README.md (74%)
 rename intro.js => praktikum-1/intro.js (100%)
 rename package.json => praktikum-1/package.json (100%)
 rename {tugas => praktikum-2/tugas}/tugas-1.js (100%)
 rename {tugas => praktikum-2/tugas}/tugas-2.js (100%)
 create mode 100644 praktikum-3/01-function-dasar.js
 create mode 100644 praktikum-3/02-arrow-callback.js
 create mode 100644 praktikum-3/03-array-dasar.js
 create mode 100644 praktikum-3/04-array-method.js
 create mode 100644 praktikum-3/05-rekursi.js
 create mode 100644 praktikum-3/package.json
 create mode 100644 praktikum-3/tugas/tugas-1.js
 create mode 100644 praktikum-3/tugas/tugas-2.js
PS C:\2025573010141_praktikum_algoritma_struktur_data\praktikum-3> git push
Enumerating objects: 19, done.
Counting objects: 100% (19/19), done.
Delta compression using up to 8 threads
Compressing objects: 100% (16/16), done.
Writing objects: 100% (16/16), 5.26 KiB | 538.00 KiB/s, done.
Total 16 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/Zafiramalmaulana/2025573010141_praktikum_algoritma_struktur_data
```

Setelah itu buka browser dan lihat pada akun github.



BAB IV
PENUTUP

4.1 KESIMPULAN

Berdasarkan praktikum yang telah saya lakukan, dapat disimpulkan bahwa function dalam **J**avaScript merupakan konsep dasar yang sangat penting dalam pemrograman. Function digunakan untuk mengelompokkan kode agar lebih terstruktur, mudah dibaca, dan dapat digunakan kembali tanpa harus menulis ulang kode yang sama.

Dalam praktikum ini, saya telah mempelajari cara membuat function, baik menggunakan deklarasi function maupun expression. Selain itu, saya juga memahami bagaimana function dapat menerima parameter sebagai input dan mengembalikan nilai (*return*) sebagai output. Hal ini sangat membantu dalam membuat program menjadi lebih fleksibel dan efisien.

Saya juga mempelajari bagaimana cara memanggil function serta perannya dalam mengurangi duplikasi kode. Dengan adanya function, program dapat dibuat lebih modular, sehingga memudahkan dalam proses pengembangan maupun debugging. Melalui praktikum ini, saya memahami bahwa penggunaan function sangat penting dalam membangun logika program yang baik. Function memungkinkan pengembang untuk membagi program menjadi bagian-bagian kecil yang lebih terorganisir.Dengan demikian, penguasaan function menjadi dasar penting dalam mempelajari JavaScript lebih lanjut. Function berperan dalam mengatur alur logika program, sedangkan HTML digunakan untuk struktur dan CSS untuk tampilan. Praktikum ini memberikan pemahaman awal yang kuat dalam membangun program yang efisien, terstruktur, dan mudah dikembangkan.

4.2 SARAN

Berdasarkan praktikum yang telah dilaksanakan, terdapat beberapa saran yang diharapkan dapat meningkatkan pemahaman dan kualitas pembelajaran di masa mendatang:

- Perbanyak latihan membuat *function* agar lebih memahami logika program.
- Pelajari materi secara bertahap dari dasar hingga lanjutan.
- Disarankan untuk rutin berlatih membuat dan menggunakan function agar lebih memahami alur logika program.
- Pelajari konsep dasar secara bertahap, mulai dari pembuatan function, parameter, hingga *return value*.